

Algorithmic Supervision of High-Frequency Financial Flows:

A Permissioned Ledger Simulation for AML Compliance in Nepal

Dikshant Pandey
Kathmandu, Nepal

January 30, 2026

Abstract

Abstract. This research presents the *Nepal Compliance Ledger (NCL)*, a regulatory technology prototype for supervising high-frequency financial flows before settlement. The system combines an **Agent-Based Model (ABM)**, a sliding-window velocity detector, bounded graph-cycle detection, and a permissioned ledger audit layer with Merkle roots, hash-linked blocks, policy binding, and simulated validator quorum signatures. The upgraded implementation simulates $N = 10,000$ autonomous actors and processes 100,000 transactions. The run produced 38,566 true positives, 39 false positives, 6,601 false negatives, and 54,794 true negatives, achieving a detection precision of **99.90%**, recall of **85.39%**, and an F1 score of **0.9207**. The ledger sealed 196 auditable blocks with intact chain verification, showing how algorithmic supervision and tamper-evident audit infrastructure can be combined in a reproducible AML compliance simulation.

1 Introduction

AML supervision often suffers from settlement latency: suspicious flows may only be discovered after money has already moved across institutions. In Nepal and similar financial environments, *structuring* or smurfing remains a practical evasion strategy, where a larger illicit transfer is fragmented into many smaller payments below scalar reporting thresholds.

This research treats the monitoring problem as a combination of velocity, graph topology, and auditability. Instead of asking only whether a single transaction exceeds a limit, NCL evaluates whether an actor is transacting too frequently inside a rolling time window and whether a new transfer closes a short circular path. The ledger layer records compliance decisions in a hash-chained permissioned structure so participating banks and regulators can verify the audit trail.

The velocity of funds can be represented as:

$$V = \frac{\sum_{i=1}^n \Delta \text{Amount}_i}{\Delta \text{Time}} \quad (1)$$

The core hypothesis is that a permissioned compliance environment can enforce velocity and topology constraints before settlement, reducing the delay between suspicious behavior and supervisory response.

2 Methodology

2.1 Architectural Simulation

The Hub-and-Spoke simulation environment models the transactional logic of a permissioned banking network. It consists of four operational layers:

1. **User Interface Layer:** Simulates mobile banking transaction requests.
2. **Compliance Brain:** Runs identity validation, velocity checks, and graph-cycle detection.
3. **Immutable Ledger Layer:** Records clean and flagged decisions in a hash-linked audit structure.
4. **Regulator Output Layer:** Provides a real-time supervisory feed for audit review.

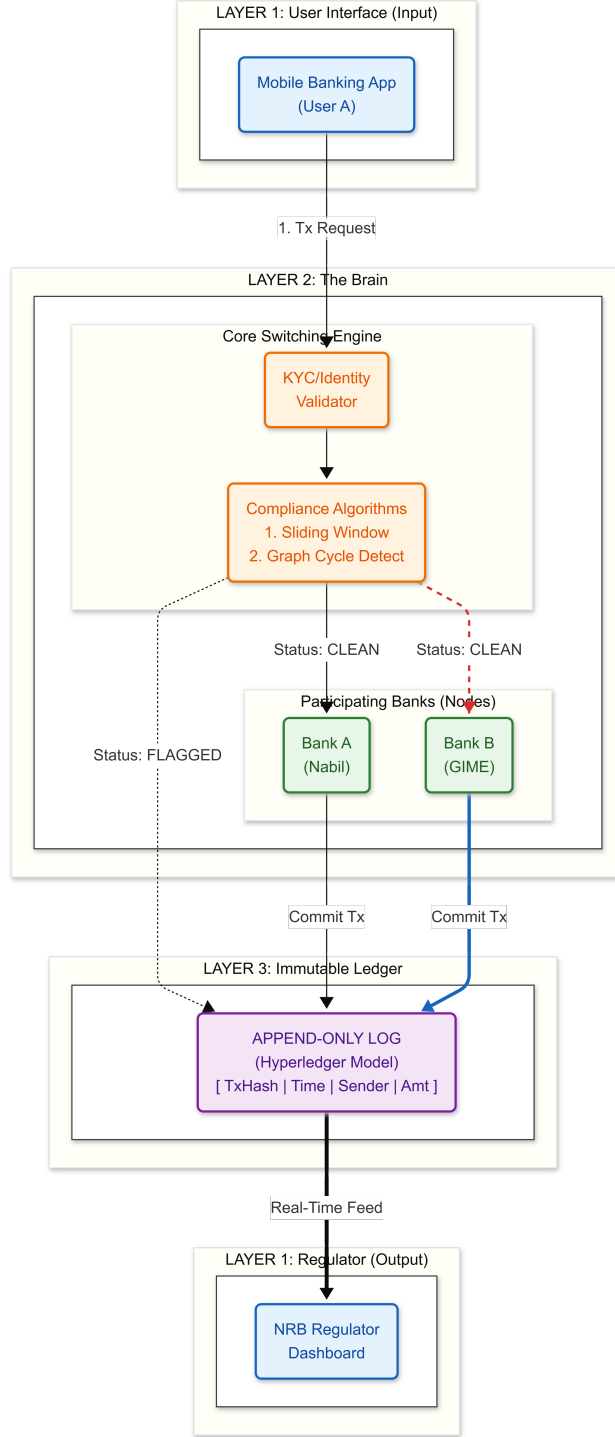


Figure 1: System Architecture: The Hub-and-Spoke Model

2.2 Algorithmic Logic 1: Sliding Window Protocol

To address the weakness of fixed-window counters, the NCL architecture implements a *sliding-window log*. For each sender, the engine keeps a queue of recent transaction timestamps and prunes

values outside the policy window.

$$\text{Count}(a, T_{\text{now}}) = \sum_{i=1}^n \mathbb{I}(T_{\text{now}} - t_i < W) \quad (2)$$

where $\mathbb{I}$ is the indicator function, a is the sender, and $W = 300$ s is the window size. A transaction is flagged when the sender's count reaches the threshold $\tau = 5$ within the active window.

2.3 Algorithmic Logic 2: Graph Cycle Detection

Scalar AML checks cannot reliably detect *layering*, where funds are routed through multiple accounts to obscure origin. NCL models transactions as a directed graph $G = (V, E)$. For a proposed transaction $e(u, v)$, the system checks whether a short path already exists from v back to u :

$$v \rightarrow \dots \rightarrow u \quad \text{where} \quad |P| \leq k \quad (3)$$

If funds return to the origin node u within $k = 3$ hops, the topology is flagged as a circular trading ring. The implementation also uses a 900 s graph memory window so the detector focuses on recent layering behavior.

Algorithm 2 Recursive Cycle Detection (DFS)

```

1: function DETECTCYCLE(origin, current, depth)
2:   if current == origin AND depth > 0 then
3:     return TRUE                                     ▷ Cycle found
4:   end if
5:   if depth ≥ MAX_DEPTH then
6:     return FALSE
7:   end if
8:   for each neighbor in AdjacencyList[current] do
9:     if DETECTCYCLE(origin, neighbor, depth + 1) then
10:      return TRUE
11:    end if
12:  end for
13:  return FALSE
14: end function

```

2.4 Algorithmic Logic 3: Permissioned Ledger Verification

The advanced NCL prototype upgrades the earlier append-only log into a permissioned block ledger. Every compliance decision is converted into a canonical audit record and batched into blocks of 512 records. Each block header contains the previous block hash, Merkle root, policy hash, proposer node, transaction count, and validator signatures.

For block B_j , the chain commitment is:

$$H(B_j) = \text{SHA256}(\text{index}_j \parallel \text{prevHash}_j \parallel \text{merkleRoot}_j \parallel \text{policyHash}_j) \quad (4)$$

The verifier accepts a run only when all previous-hash links, Merkle roots, block hashes, and simulated validator signatures satisfy the policy quorum of 4 out of 5 validators.

3 Experimental Results

3.1 Simulation Setup

The experiment uses a deterministic ABM configuration with $N = 10,000$ autonomous agents and 100,000 generated transactions:

- **Honest Agents (95%):** Stochastic legitimate transactions across participating banks.
- **Malicious Agents (5%):** Programmed to execute structuring and circular layering behaviors.
- **Ledger Configuration:** 512 records per block, SHA-256 hashing, Merkle roots, and simulated 4/5 validator quorum.

3.2 Findings

The simulation processed 100,000 transactions. As shown in Figure 2, the engine maintained high precision while sealing the audit trail into permissioned blocks.

```
--- INITIALIZING AGENT-BASED MODEL (10000 AGENTS) ---
>>> SIMULATION ENGINE STARTED...
>>> PROCESSING 100000 TRANSACTIONS...
>>> SEALING MERKLE BLOCKS WITH 4/5 VALIDATOR QUORUM...

=====
AGENT-BASED SIMULATION RESULTS (N=10000)
=====
Total Transactions Processed: 100,000
-----
[CONFUSION MATRIX]
> True Positives (Blocked Criminals): 38,566
> False Positives (Blocked Honest): 39
> False Negatives (Missed Criminals): 6,601
> True Negatives (Allowed Honest): 54,794
-----
[PERFORMANCE METRICS]
> Precision: 0.9990
> Recall: 0.8539
> F1 Score: 0.9207
> Throughput: 102,948.19 tx/s
-----
[PERMISSIONED LEDGER]
> Sealed Blocks: 196
> Block Size: 512
> Validator Quorum: 4/5
> Last Block Hash: adc9d4ae8a83d0c854769040...
> Chain Integrity: INTACT
=====
```

Figure 2: Output Logs from Agent-Based Simulation ($N = 10,000$)

Metric	Value	Interpretation
Precision	0.9990	Very low customer-friction rate
Recall	0.8539	Strong attack capture across synthetic behaviors
F1 Score	0.9207	Balanced detection performance
Throughput	102,948 tx/s	Laptop simulation processing speed
Sealed Blocks	196	Permissioned ledger audit batches
Chain Integrity	Intact	Hash, Merkle, and quorum checks passed

Table 1: NCL 100k transaction simulation summary.

The confusion matrix from the reproducible run was:

- **True Positives:** 38,566
- **False Positives:** 39
- **False Negatives:** 6,601
- **True Negatives:** 54,794

4 Discussion

4.1 Architectural Justification: Why a Permissioned Ledger?

A centralized database can record transaction decisions, but it also concentrates trust in a single administrator or institution. NCL uses a permissioned ledger model because AML supervision is multi-institutional: banks and regulators need a shared audit object whose integrity can be independently verified. In the prototype, each block binds transaction decisions to a previous hash, Merkle root, policy hash, and validator quorum, making later manipulation detectable.

4.2 Detection Behavior

Most detected malicious activity was captured by the sliding-window velocity protocol, which is expected because structuring is represented as repeated low-value transfers. The graph-cycle detector contributes an additional layer for circular trading rings. This two-part approach keeps the policy interpretable: investigators can see whether a transaction was flagged because of excessive velocity or because it completed a suspicious path.

4.3 Limitations & Future Work

The current implementation is a deterministic simulation, not a production banking system. Validator signatures are simulated rather than backed by real institutional keys or HSMs. The ABM uses synthetic behavior distributions that would require calibration against legally shareable banking data. Future work should evaluate stochastic windows, richer graph anomaly scoring, and privacy-preserving regulatory data sharing.

5 Conclusion

The Nepal Compliance Ledger simulation provides empirical evidence that algorithmic AML supervision can be modeled as a pre-settlement compliance problem. By combining sliding-window velocity checks, graph-cycle detection, and a permissioned audit ledger, NCL demonstrates how high-frequency financial flows can be filtered and recorded in a tamper-evident system. The upgraded 100k-transaction run shows strong precision, reproducible ledger integrity, and a clear path for expanding the prototype into larger regulatory technology experiments.

References

- [1] FATF, *International Standards on Combating Money Laundering and the Financing of Terrorism and Proliferation*.
- [2] Nepal Rastra Bank, *Unified Directives and AML/CFT Supervisory Guidance*.
- [3] Androulaki, E. et al., “Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains,” EuroSys, 2018.
- [4] Nakamoto, S., “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [5] Savage, D. et al., “Detection of Money Laundering Groups Using Supervised Learning in Networks,” arXiv:1608.00708, 2016.